

Session 02: Machine Learning vs Deep Learning and the End-to-End AI Workflow

This session establishes a clear conceptual understanding of the relationship between Machine Learning (ML), Deep Learning (DL), and Artificial Intelligence as a whole. It then introduces the standard AI workflow and contextualizes it within agricultural applications.

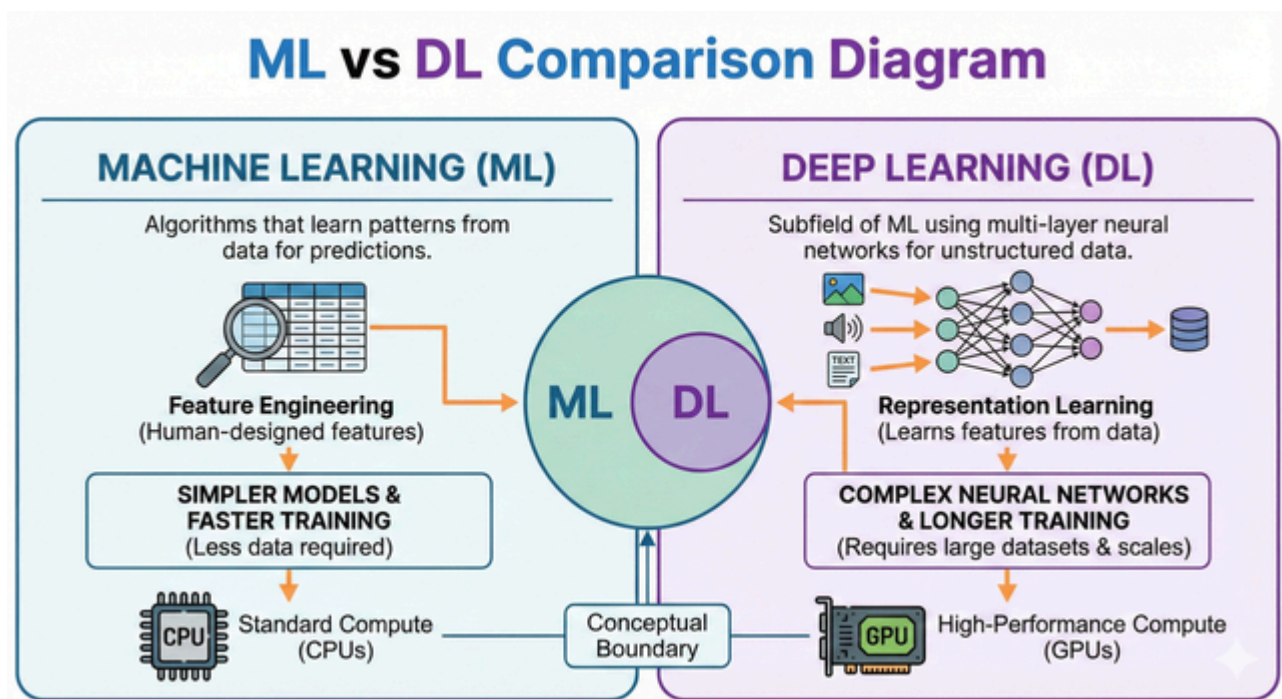
1. Conceptual Boundary Between ML and DL

Machine Learning encompasses algorithms that learn patterns from data to make predictions or decisions. These models typically rely on structured features engineered by humans.

Deep Learning is a specialized subfield of ML based on neural networks with multiple layers capable of learning hierarchical representations directly from data, particularly unstructured data such as images, audio, and text.

Key distinctions:

- ML relies more on feature engineering; DL relies on representation learning.
- ML models are often simpler, faster to train, and require less data.
- DL models scale better with large datasets and complex tasks.
- DL depends heavily on GPUs and high-performance compute environments.



2. Taxonomy of Machine Learning Paradigms

A structured map of ML categories.

Supervised Learning

Models learn from labeled data.

- Regression
- Classification
- Practical applications: yield prediction, disease detection

Unsupervised Learning

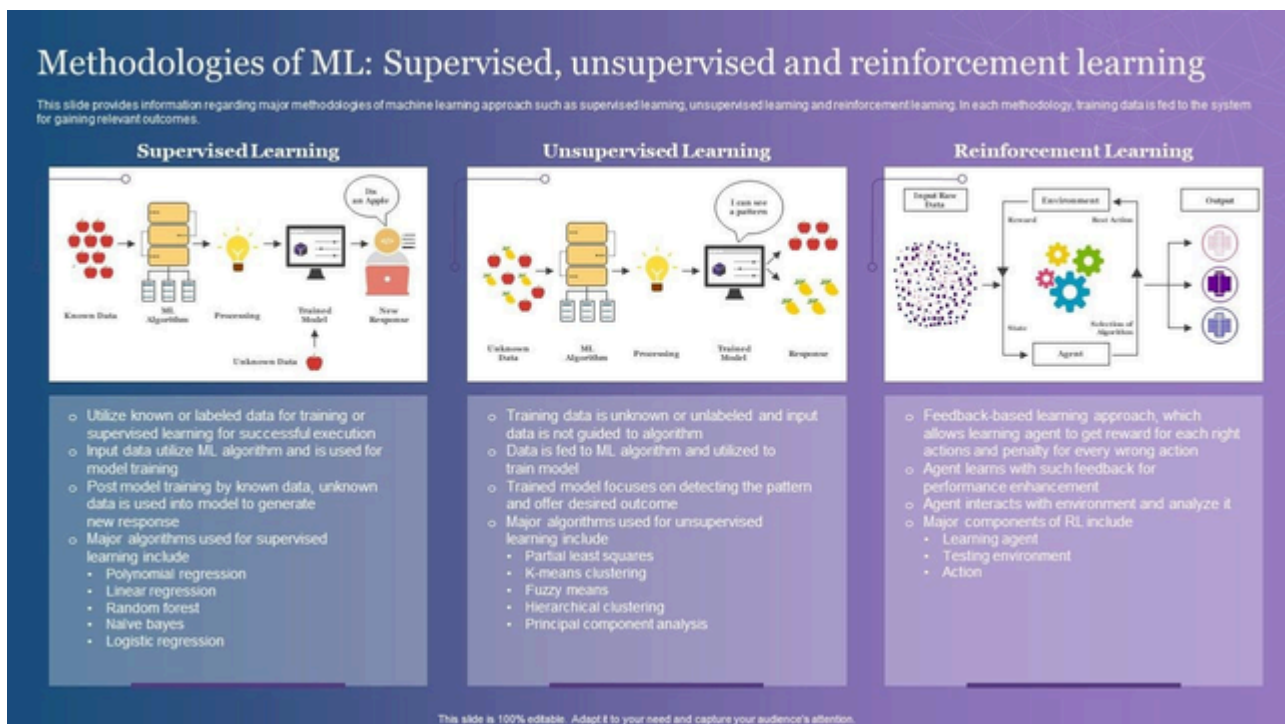
Models discover structure in unlabeled data.

- Clustering
- Dimensionality reduction
- Applications: soil type grouping, field segmentation

Reinforcement Learning

Agents learn by interacting with an environment and receiving rewards.

- Applications: autonomous drones, irrigation automation



3. Deep Learning Fundamentals

Core ideas underpinning modern DL systems.

Neural Networks

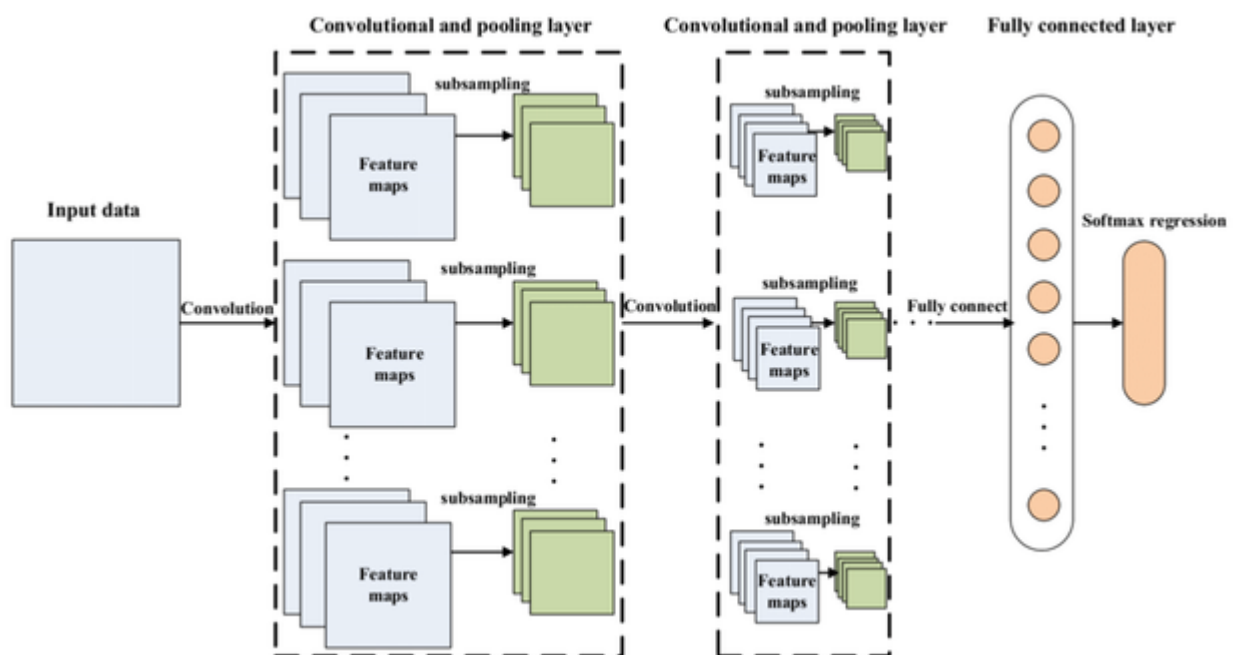
Layers of interconnected units (neurons) that transform inputs through learned weights.

Key Concepts

- Activation functions
- Loss functions
- Optimization (SGD, Adam)
- Training vs inference
- Overfitting and regularization

Common Architectures

- Convolutional Neural Networks (CNNs) for images
- Recurrent Neural Networks (RNNs) and LSTMs for sequences
- Transformers for general-purpose modeling

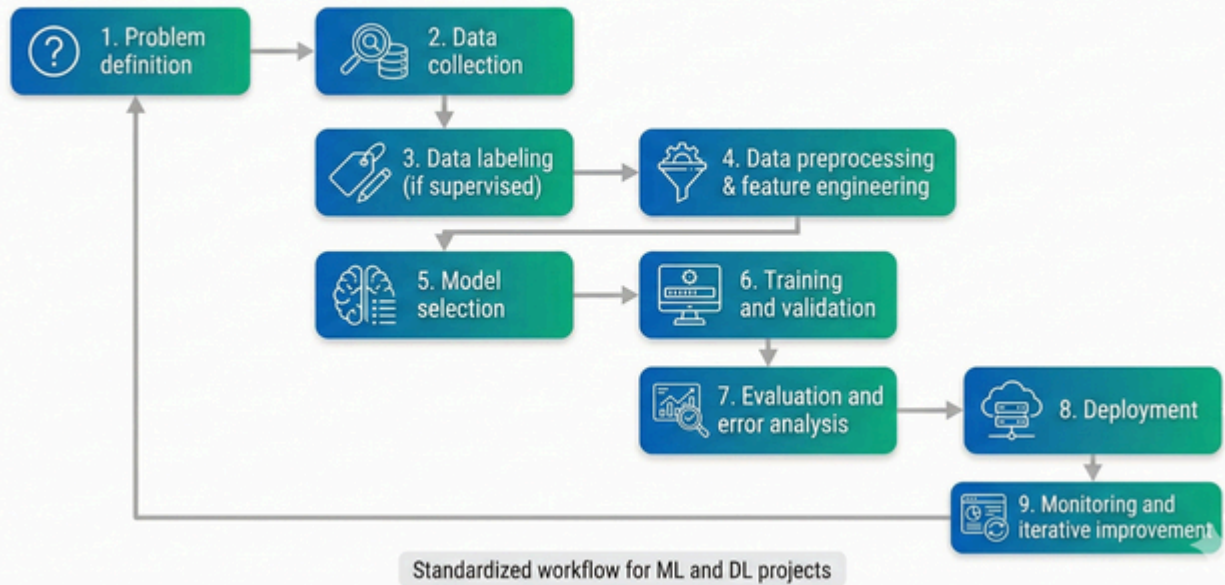


4. End-to-End AI Workflow

A standardized workflow used in both ML and DL projects.

1. Problem definition
2. Data collection
3. Data labeling (if supervised)
4. Data preprocessing and feature engineering
5. Model selection
6. Training and validation
7. Evaluation and error analysis
8. Deployment
9. Monitoring and iterative improvement

End-to-end AI Workflow Diagram



5. AI Workflow in Agriculture

Applying the general AI workflow to domain-specific agricultural problems.

Example Contexts

- Crop disease detection using images
- Yield forecasting using numerical and environmental data
- Soil classification using satellite data

Adaptations for Agriculture

- Data collection may involve drones, sensors, or satellite imagery.
- Labeling often requires agronomists or domain specialists.
- Environmental variability makes model generalization more difficult.
- Deployment may occur on edge devices in remote environments.

Challenge Exercise

Use the **MNIST** handwritten-digit dataset to explore how **spatial structure** affects what a neural network can learn. You will train two identical feedforward (fully connected) networks — one on the original images and one on a **pixel-shuffled** version of the same data — then compare the results.

Install the extra dependency if needed:

```
poetry add torchvision
```

Part 1 — Baseline: MNIST with a normal neural network

1. Load MNIST with `torchvision.datasets.MNIST` (training and test splits).
2. Train the model on the **original** training set and evaluate accuracy on the **original** test set.
3. Visualize a few test images alongside the model's predictions.

Record the test accuracy — this is your **baseline**.

Part 2 — Stable pixel shuffle (same permutation for the whole dataset)

The goal is to destroy the **spatial layout** of each image while keeping pixel **values** intact.

1. Generate **one** random permutation
2. Apply **exactly this same permutation** to **every** image in both the training set and the test set
 - After shuffling, pixel position `1` might now hold what was pixel `13` on every image in the dataset. To a human, the digits will look like random noise — but the reordering is **consistent** across all samples.
3. Visualize a few shuffled images so you can see how unrecognizable they look compared to the originals.

Important: use one permutation for the entire dataset. Do **not** shuffle pixels differently per image — that would be a different experiment.

Part 3 — Train a second model on the shuffled data

1. Build a **new** network with the **same architecture** as in Part 1 (or reset the weights of the first model).
2. Train it on the **shuffled** training set.
3. Evaluate on the **shuffled** test set (apply the same `perm` before inference).

Record this test accuracy and compare it to your baseline.

Part 4 — Report your observations

Write a short summary (a few paragraphs or bullet points) covering:

Question	What to discuss
Baseline accuracy	What accuracy did the model reach on original MNIST?
Shuffled accuracy	What accuracy did the second model reach on shuffled MNIST?
Visual difference	How do original vs shuffled images look to you?
Why the results differ (or don't)	A fully connected network treats each input dimension independently. Does destroying 2D spatial layout hurt as much as you expected when the same shuffle is applied everywhere?
Key takeaway	What does this tell you about the difference between how humans recognize digits and how a plain neural network learns from pixel values?

Submit your notebook or script along with your written observations.